

LAPORAN PRATIUM STRUKTUR DATA

IMPLEMENTASI ALGORITMA SORTING (BUBBLE, SELECTION, INSERTION)

DALAM JAVA DENGAN VISUALISASI LANGSUNG

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : Dr. WAHYUDI, S.T, M.T

ASISTEN LABORATORIUM : Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum “Implementasi Algoritma Sorting (Bubble Sort, Selection Sort, Insertion Sort) dalam Java dengan Visualisasi Langsung” ini dapat diselesaikan dengan baik. Saya ucapkan terima kasih sebesar-besarnya kepada Dr. Wahyudi, S.T, M.T selaku dosen pengampu mata kuliah Struktur Data, serta kepada asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum Pekan 7 yang bertujuan untuk memahami dan mengimplementasikan tiga algoritma sorting dasar: *Bubble Sort*, *Selection Sort*, dan *Insertion Sort*. Berbeda dengan praktikum sebelumnya yang fokus pada struktur data *Double Linked List*, praktikum kali ini menekankan pada pengurutan data dalam *array* serta pembuatan antarmuka pengguna (*graphical user interface*) untuk memvisualisasikan proses sorting langkah per langkah. Pembuatan GUI dengan *Swing* memungkinkan pengguna untuk melihat secara langsung bagaimana elemen-elemen data dibandingkan dan dipindahkan hingga terurut.

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna. Namun, dengan segala kemampuan dan pengetahuan yang dimiliki, saya telah berupaya agar laporan ini dapat selesai dengan baik. Oleh karenanya, saya dengan rendah hati menerima saran, masukan, dan kritikan guna penyempurnaan laporan ini.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Manfaat	4
BAB II PEMBAHASAN.....	5
2.1 Konsep Double Linked List dalam Java	5
2.2 Langkah pengerjaan	6
2.3 Latihan	11
BAB III KESIMPULAN.....	15
3.1 Kesimpulan	15
3.2 Saran	15
DAFTAR PUSTAKA.....	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum minggu-minggu sebelumnya, kita telah mempelajari struktur data linier seperti *array* dan *linked list* (baik *single* maupun *double*). Namun, menyimpan data saja tidak cukup; seringkali kita perlu mengatur data tersebut agar terurut berdasarkan kriteria tertentu, misalnya mengurutkan nilai mahasiswa dari yang tertinggi, mengurutkan nama secara alfabetis, atau mengurutkan tanggal transaksi. Proses pengurutan ini disebut *sorting*.

Sorting adalah salah satu fondasi penting dalam algoritma dan struktur data karena banyak algoritma lain (seperti pencarian biner) hanya bisa bekerja efisien jika data sudah terurut. Terdapat banyak algoritma *sorting*, masing-masing dengan karakteristik kompleksitas waktu dan ruang yang berbeda. Pada praktikum Pekan 7 ini, kita fokus pada tiga algoritma *sorting* sederhana namun fundamental:

1. Bubble Sort – Menggelembungkan elemen terbesar ke ujung secara berulang.
2. Selection Sort – Memilih elemen terkecil dan menukarnya ke posisi awal secara bertahap.
3. Insertion Sort – Menyisipkan setiap elemen ke posisi yang tepat pada bagian array yang sudah terurut.

Selain implementasi algoritma dalam bentuk *command line*, praktikum ini juga menuntut pembuatan program visual interaktif menggunakan *Java Swing*. GUI ini memungkinkan pengguna untuk melihat proses *sorting* langkah per langkah, sehingga pemahaman tentang mekanisme perbandingan dan pertukaran elemen menjadi lebih intuitif. Sebagai studi kasus, kita akan membuat aplikasi pengurutan data mahasiswa (berdasarkan nama) dengan pilihan tiga algoritma, tombol langkah, reset, serta log proses.

1.2 Tujuan

1. Memahami konsep dan mekanisme tiga algoritma sorting dasar: Bubble Sort, Selection Sort, dan Insertion Sort.
2. Mampu mengimplementasikan ketiga algoritma sorting tersebut dalam bahasa Java untuk tipe data primitif (int) dan objek (Mahasiswa).
3. Mampu membuat program visual interaktif dengan Java Swing untuk menampilkan proses sorting langkah demi langkah.
4. Memahami perbedaan kompleksitas waktu dan karakteristik setiap algoritma sorting.
5. Menerapkan sorting pada studi kasus pengurutan data mahasiswa berdasarkan nama (string) secara ascending.
6. Mengelola *state* algoritma agar dapat menjalankan sorting secara bertahap (*step-by-step*) dengan tombol.

1.3 Manfaat

1. Mahasiswa memahami tiga algoritma sorting klasik beserta kelebihan dan kekurangannya.
2. Mahasiswa terampil mengimplementasikan sorting pada *array of objects* dengan *custom comparator* (perbandingan string).
3. Mahasiswa mampu membangun antarmuka pengguna untuk visualisasi algoritma, yang berguna untuk pembelajaran dan debugging.
4. Mahasiswa memperoleh pengalaman dalam mengelola *state* (variabel seperti *i*, *j*, *minIdx*) yang tersimpan di antara langkah-langkah eksekusi, suatu keterampilan penting dalam pemrograman *event-driven*.

BAB II

PEMBAHASAN

2.1 Konsep Single Linked List dalam Java

Sorting (pengurutan) adalah proses mengatur sekumpulan data dalam urutan tertentu (misal ascending atau descending) berdasarkan suatu *key*. Tiga algoritma yang dipraktikkan:

a. Bubble Sort

Bubble Sort bekerja dengan membandingkan elemen bersebelahan dan menukarnya jika tidak sesuai urutan. Proses ini diulang sebanyak $n-1$ kali, di mana setiap iterasi akan "menggelembungkan" elemen terbesar ke posisi akhir. Kompleksitas waktu $O(n^2)$ dalam kasus rata-rata dan terburuk, $O(n)$ jika array sudah terurut (dengan optimasi *swapped flag*). Karena sederhana, Bubble Sort cocok untuk data kecil.

b. Selection Sort

Selection Sort membagi array menjadi dua bagian: terurut (di kiri) dan belum terurut (di kanan). Setiap langkah mencari elemen terkecil dari bagian belum terurut, lalu menukarnya dengan elemen pertama bagian belum terurut. Kompleksitas waktu selalu $O(n^2)$ terlepas dari kondisi data, tetapi jumlah pertukaran data minimal (maksimal $n-1$ kali). Stabil? Tidak, karena pertukaran dapat mengubah urutan relatif elemen yang sama.

c. Insertion Sort

Insertion Sort mirip dengan cara orang mengatur kartu: ambil satu kartu, lalu sisipkan ke posisi yang tepat di antara kartu yang sudah terurut. Algoritma ini berjalan dengan baik pada data yang hampir terurut dan memiliki kompleksitas $O(n)$ pada kasus terbaik. Kompleksitas rata-rata dan terburuk $O(n^2)$. Keunggulan: bersifat *stable* (mempertahankan urutan relatif elemen sama) dan *in-place*.

Dalam praktikum ini, semua algoritma diimplementasikan untuk mengurutkan berdasarkan nama mahasiswa secara ascending (A-Z) dengan membandingkan string menggunakan `compareToIgnoreCase`.

2.2 Langkah pengerjaan

1. Program BubbleSort_2511532029

Program:

```
1 package penast7_2511532029;
2 public class BubbleSort_2511532029 {
3
4     public static void bubbleSort_2029(int[] arr_2029) {
5         int n_2029 = arr_2029.length;
6         for (int i_2029 = 0; i_2029 < n_2029 - 1; i_2029++) {
7             for (int j_2029 = 0; j_2029 < n_2029 - i_2029; j_2029++) {
8                 if (arr_2029[j_2029] > arr_2029[j_2029 + 1]) {
9                     int temp_2029 = arr_2029[j_2029];
10                    arr_2029[j_2029] = arr_2029[j_2029 + 1];
11                    arr_2029[j_2029 + 1] = temp_2029;
12                }
13            }
14        }
15    }
16
17    public static void main(String[] args) {
18        int[] arr_2029 = {23, 78, 45, 8, 32, 56, 1};
19        int n_2029 = arr_2029.length;
20
21        System.out.print("Array yang belum terurut: ");
22        for (int i_2029 = 0; i_2029 < n_2029; i_2029++) {
23            System.out.print(arr_2029[i_2029] + " ");
24        }
25        System.out.println();
26
27        bubbleSort_2029(arr_2029);
28
29        System.out.print("Array yang sudah terurut: ");
30        for (int i_2029 = 0; i_2029 < n_2029; i_2029++) {
31            System.out.print(arr_2029[i_2029] + " ");
32        }
33        System.out.println();
34    }
35 }
```

Gambar 2.1

Penjelasan program:

Program ini mengimplementasikan *Bubble Sort* untuk array integer.

Fungsi bubbleSort_2029 menggunakan dua perulangan bersarang:

- Perulangan luar (i_2029) melambangkan jumlah *pass* (maksimal n-1).
- Perulangan dalam (j_2029) membandingkan arr[j] dengan arr[j+1]. Batas perulangan dalam berkurang setiap kali karena elemen terbesar sudah berada di akhir.
- Jika elemen kiri lebih besar dari kanan, maka ditukar menggunakan variabel sementara.

Pada main, array awal {23, 78, 45, 8, 32, 56, 1} dicetak, kemudian diurutkan, dan hasil akhir dicetak. Kompleksitas $O(n^2)$ namun mudah diimplementasikan.

Output:

```
<terminated> BubleSort_2511532029 [Java Application] C
```

```
Array yang belum terurut: 23 78 45 8 32 56 1
```

```
Array yang sudah terurut: 1 8 23 32 45 56 78
```

2. SelectionSort_2511532029 Program:

```
1 package pekariv_2511532029;
2
3 public class SelectionSort_2511532029 {
4     public static void selectionSort_2029(int[] arr_2029) {
5         int n_2029 = arr_2029.length;
6         for (int i_2029 = 0; i_2029 < n_2029 - 1; i_2029++) {
7             int minIndex_2029 = i_2029;
8             for (int j_2029 = i_2029 + 1; j_2029 < n_2029; j_2029++)
9                 if (arr_2029[j_2029] < arr_2029[minIndex_2029]) {
10                     minIndex_2029 = j_2029;
11                 }
12             // Tukar elemen
13             int temp_2029 = arr_2029[i_2029];
14             arr_2029[i_2029] = arr_2029[minIndex_2029];
15             arr_2029[minIndex_2029] = temp_2029;
16         }
17     }
18
19     public static void main(String[] args) {
20         int arr_2029[] = {23, 78, 45, 8, 32, 56, 1};
21         int n = arr_2029.length;
22
23         System.out.print("Array yang belum terurut: ");
24         for (int i_2029 = 0; i_2029 < n; i_2029++)
25             System.out.print(arr_2029[i_2029] + " ");
26         System.out.println();
27
28         selectionSort_2029(arr_2029);
29
30         System.out.print("Array yang sudah terurut: ");
31         for (int i_2029 = 0; i_2029 < n; i_2029++)
32             System.out.print(arr_2029[i_2029] + " ");
33         System.out.println();
34     }
35 }
36 }
```

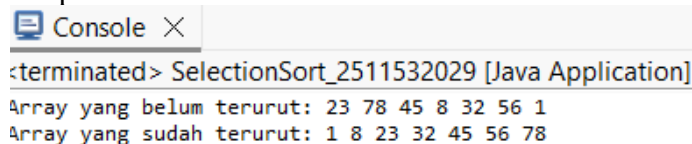
Gambar 2.2

Penjelasan program:

Selection Sort bekerja dengan membagi array menjadi dua sisi: sisi kiri (sudah terurut) dan sisi kanan (belum terurut).

- Perulangan luar (*i_2029*) menunjukkan batas sisi terurut.
- Dicari indeks elemen terkecil (*minIndex_2029*) di sisi belum terurut (dari *i_2029* hingga akhir).
- Setelah ditemukan, elemen pada posisi *i_2029* ditukar dengan elemen terkecil tersebut.
- Dengan demikian, setiap iterasi menempatkan satu elemen ke posisi yang benar. Kompleksitas tetap $O(n^2)$ karena dua perulangan, tetapi jumlah pertukaran hanya $n-1$ kali, lebih hemat daripada Bubble Sort untuk data besar.

Output:



```
<terminated> SelectionSort_2511532029 [Java Application]
Array yang belum terurut: 23 78 45 8 32 56 1
Array yang sudah terurut: 1 8 23 32 45 56 78
```

Gambar 2.3

3. Program InsertionSort_2511532029

Program:

```
1 package pekan7_2511532029;
2
3 public class InsertionSort_2511532029 {
4     public static void insertionSort_2029(int[] arr_2029) {
5         int n_2029 = arr_2029.length;
6         // Mulai dari indeks 1 karena elemen pertama dianggap sudah terurut
7         for (int i_2029 = 1; i_2029 < n_2029; i_2029++) {
8             int key_2029 = arr_2029[i_2029];
9             int j_2029 = i_2029 - 1;
10            // Geser elemen yang lebih besar dari key ke kanan
11            while (j_2029 >= 0 && arr_2029[j_2029] > key_2029) {
12                arr_2029[j_2029+1] = arr_2029[j_2029];
13                j_2029--;
14            }
15            // Tempatkan key pada posisi yang tepat
16            arr_2029[j_2029+1] = key_2029;
17        }
18    }
19
20    public static void main(String[] args) {
21        int arr_2029[] = {23, 78, 45, 8, 32, 56, 1};
22        int n_2029 = arr_2029.length;
23
24        System.out.print("Array yang belum terurut: ");
25        for (int i_2029 = 0; i_2029 < n_2029; i_2029++) {
26            System.out.print(arr_2029[i_2029] + " ");
27        }
28        System.out.println();
29
30        insertionSort_2029(arr_2029);
31
32        System.out.print("Array yang sudah terurut: ");
33        for (int i_2029 = 0; i_2029 < n_2029; i_2029++) {
34            System.out.print(arr_2029[i_2029] + " ");
35        }
36        System.out.println();
37    }
38 }
```

Gambar 2.4

Penjelasan program:

Insertion Sort membangun array terurut secara bertahap:

- Mulai dari indeks 1 (anggap indeks 0 sudah terurut).
- Ambil key = arr[i].
- Bandingkan key dengan elemen di sebelah kirinya (j). Selama elemen kiri lebih besar dari key, geser elemen tersebut ke kanan (arr[j+1] = arr[j]).
- Setelah perulangan selesai, tempatkan key pada posisi j+1 yang tepat.
- Proses diulang untuk i berikutnya.

Kompleksitas terbaik $O(n)$ (jika sudah terurut), rata-rata $O(n^2)$. Algoritma ini stabil dan efisien untuk data kecil atau hampir terurut.

Output:

```
<terminated> InsertionSort_2511532029 [Java Application]
Array yang belum terurut: 23 78 45 8 32 56 1
Array yang sudah terurut: 1 8 23 32 45 56 78
```

Gambar 2.5

4. Program SortingGUI_2511532029

Program:

[illegible]

Gambar 2.6

Penjelasan program:

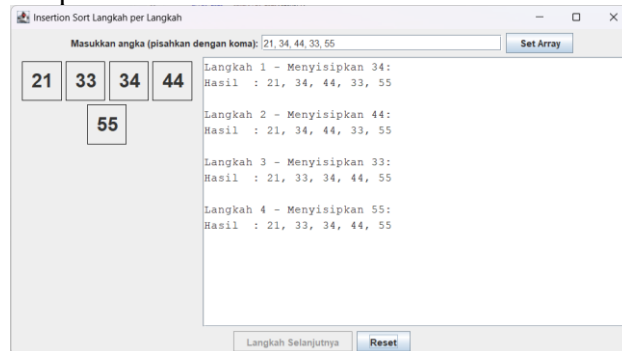
Program ini adalah **visualisasi interaktif Insertion Sort** menggunakan *Java Swing*. Pengguna dapat memasukkan array angka (dipisah koma), lalu menekan tombol "Langkah Selanjutnya" untuk menjalankan satu demi satu langkah penyisipan.

Komponen utama:

- JTextField untuk input array.
- JButton "Set Array" untuk membaca input dan mereset sorting.
- JPanel berisi JLabel yang merepresentasikan setiap elemen array dengan border dan ukuran tetap.
- JTextArea untuk mencatat log setiap langkah.

- Tombol "Langkah Selanjutnya" memanggil performStep_2029() yang menjalankan **satu iterasi dari perulangan luar** Insertion Sort (memproses satu key).
- State disimpan dalam variabel i_2029 (indeks yang sedang diproses) dan sorting_2029.
Keunggulan program ini adalah **pembelajaran visual**: mahasiswa dapat melihat perubahan posisi elemen secara langsung dan memahami proses penggeseran serta penyisipan.

Output:



Gambar 2.7

2.3 Latihan

Deskripsi Program:

Program ini adalah aplikasi GUI lengkap untuk mengelola data mahasiswa dan mengurutkannya dengan tiga algoritma sorting (Bubble, Selection, Insertion) secara step-by-step. Pengguna dapat menambah, menghapus, dan memilih mahasiswa, lalu memilih algoritma sorting, serta melangkah satu per satu untuk melihat proses perbandingan dan pertukaran. Fitur ini memanfaatkan kemampuan menyimpan *state* algoritma di antara event tombol.

1. Program Mahasiswa_2511532029

Program:

```
1 package pekan7_2511532029;
2
3 public class Mahasiswa_2511532029 {
4     private String nama_2029;
5     private String nim_2029;
6     private String prodi_2029;
7
8     public Mahasiswa_2511532029(String nama, String nim, String prodi) {
9         this.nama_2029 = nama;
10        this.nim_2029 = nim;
11        this.prodi_2029 = prodi;
12    }
13
14    public String getNama_2029() { return nama_2029; }
15    public String getNim_2029() { return nim_2029; }
16    public String getProdi_2029() { return prodi_2029; }
17
18    public String toString_2029() {
19        return nama_2029 + " (" + nim_2029 + ")";
20    }
21 }
```

Gambar 2.8

a) Deskripsi program Mahasiswa_2511532029 :

Kelas ini merepresentasikan entitas mahasiswa dengan atribut nama, nim, prodi (enkapsulasi private). Disediakan *getter* dan metode *toString_2029*. Sorting nantinya akan membandingkan berdasarkan nama menggunakan *compareToIgnoreCase*.

2. Program SortingGUI_2511532029

Program:

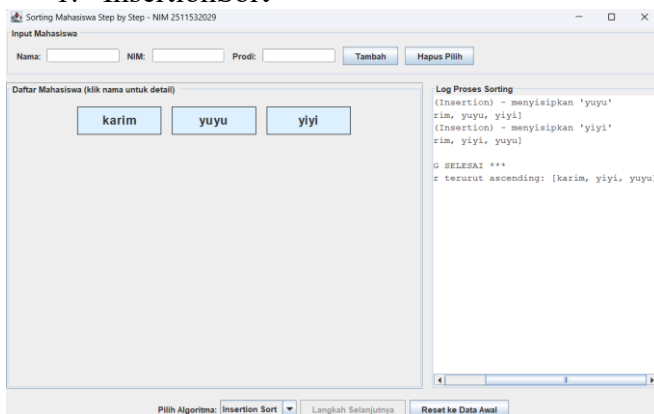
```
1 // SortingGUI_2511532029
2 // Author: 2511532029
3 // Date: 2025-11-15
4 // Description: A Java Swing application for sorting an array of integers.
5 // The application has a text area for input, a 'Sort' button, and a text area for output.
6 // It implements a simple sorting algorithm (Bubble Sort).
7
8 import javax.swing.*;
9 import java.awt.*;
10 import java.awt.event.*;
11
12 public class SortingGUI {
13     private JFrame window;
14     private JTextArea inputArea;
15     private JButton sortButton;
16     private JTextArea outputArea;
17
18     public SortingGUI() {
19         window = new JFrame("SortingGUI_2511532029");
20         window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
21         window.setSize(400, 300);
22
23         inputArea = new JTextArea(10, 40);
24         sortButton = new JButton("Sort");
25         outputArea = new JTextArea(10, 40);
26
27         window.setLayout(new BorderLayout());
28         window.add(inputArea, BorderLayout.NORTH);
29         window.add(sortButton, BorderLayout.CENTER);
30         window.add(outputArea, BorderLayout.SOUTH);
31
32         sortButton.addActionListener(new SortListener());
33
34         window.setVisible(true);
35     }
36
37     private class SortListener implements ActionListener {
38         public void actionPerformed(ActionEvent e) {
39             String input = inputArea.getText();
40             int[] array = parseInput(input);
41             sort(array);
42             String output = formatOutput(array);
43             outputArea.setText(output);
44         }
45     }
46
47     private int[] parseInput(String input) {
48         String[] tokens = input.split(",");
49         int[] array = new int[tokens.length];
50         for (int i = 0; i < tokens.length; i++) {
51             array[i] = Integer.parseInt(tokens[i].trim());
52         }
53         return array;
54     }
55
56     private void sort(int[] array) {
57         // Bubble Sort implementation
58         for (int i = 0; i < array.length - 1; i++) {
59             for (int j = 0; j < array.length - i - 1; j++) {
60                 if (array[j] > array[j + 1]) {
61                     // Swap elements
62                     int temp = array[j];
63                     array[j] = array[j + 1];
64                     array[j + 1] = temp;
65                 }
66             }
67         }
68     }
69
70     private String formatOutput(int[] array) {
71         StringBuilder sb = new StringBuilder();
72         for (int i = 0; i < array.length; i++) {
73             sb.append(array[i]);
74             if (i < array.length - 1) {
75                 sb.append(", ");
76             }
77         }
78         return sb.toString();
79     }
80 }
81
82 // End of Program
```

Gambar 2.9

- a) Program ini merupakan puncak dari praktikum karena mengintegrasikan:
- Manajemen data mahasiswa (tambah, hapus, tampil dalam bentuk label yang bisa diklik untuk detail).
 - Tiga algoritma sorting yang dapat dipilih via JComboBox.
 - Step-by-step execution dengan menyimpan *state* masing-masing algoritma dalam variabel kelas (misal insertI_2029, selectI_2029, bubbleI_2029, dll). Setiap kali tombol "Langkah Selanjutnya" ditekan, satu langkah kecil algoritma dijalankan (satu penyisipan untuk Insertion Sort, satu pencarian minimum atau satu tukar untuk Selection Sort, satu perbandingan/tukar untuk Bubble Sort).
 - Reset ke data awal (sebelum sorting) dan kemampuan berganti algoritma (data direset ke urutan awal).
 - Log area yang mencatat setiap aksi (perbandingan, pertukaran, hasil sementara).

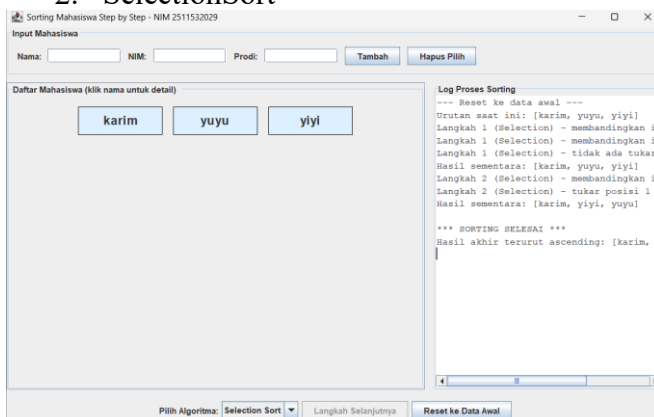
Output:

1. InsertionSort

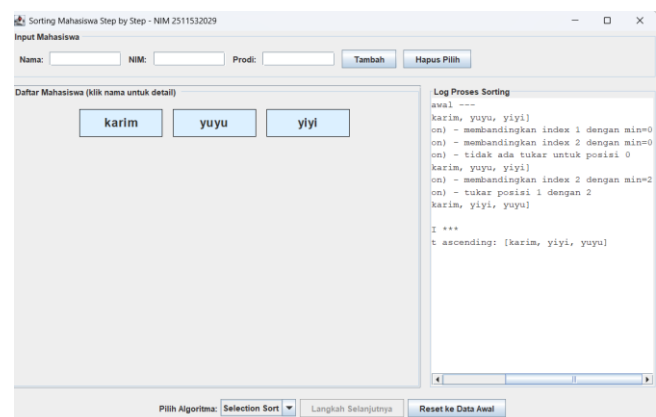


Gambar 2.10

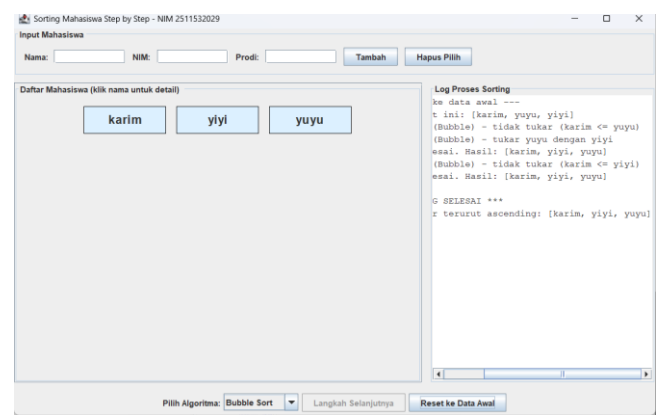
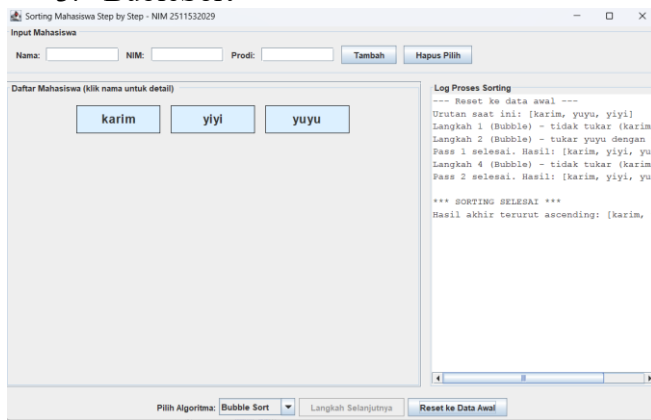
2. SelectionSort



Gambar 2.10



3. BubleSort



Gambar 2.10

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan pelaksanaan praktikum implementasi algoritma sorting (Bubble Sort, Selection Sort, dan Insertion Sort) dalam bahasa Java, dapat disimpulkan bahwa ketiga algoritma tersebut memiliki kompleksitas waktu rata-rata $O(n^2)$ sehingga paling cocok digunakan untuk dataset berukuran kecil hingga menengah. Perbedaan utama terletak pada mekanisme kerjanya: Bubble Sort menggelembungkan elemen terbesar ke ujung melalui pertukaran berulang dengan elemen tetangga, Selection Sort memilih elemen terkecil dari bagian belum terurut lalu menukarnya ke posisi awal, sedangkan Insertion Sort menyisipkan setiap elemen ke posisi yang tepat pada bagian array yang sudah terurut. Implementasi dalam bentuk command line berhasil membuktikan kebenaran algoritma pada array integer, sekaligus menjadi fondasi untuk pengembangan lebih lanjut ke dalam program berbasis antarmuka pengguna.

Studi kasus utama berupa aplikasi SortingGUI_2511532029 berhasil mengintegrasikan manajemen data mahasiswa (tambah, hapus, tampilkan), pilihan tiga algoritma sorting, serta eksekusi langkah per langkah (step-by-step) yang disertai dengan log proses dan fitur reset. Keberhasilan ini menunjukkan bahwa penyimpanan *state* algoritma (seperti variabel *i*, *j*, *minIdx*) di antara event tombol merupakan kunci untuk memvisualisasikan proses sorting secara interaktif, sehingga sangat membantu pemahaman mahasiswa tentang mekanisme perbandingan dan pertukaran data. Program ini sekaligus menjadi landasan untuk pengembangan lebih lanjut, seperti penambahan visualisasi grafik, fitur auto-step dengan pengatur kecepatan, serta penyimpanan data ke file.

3.2 Saran

1. Penggunaan visualisasi proses perbandingan dan pertukaran elemen pada algoritma sorting (seperti Bubble Sort, Selection Sort, dan Insertion Sort) melalui animasi atau mode debug akan sangat membantu mahasiswa dalam memahami alur kerja setiap algoritma secara lebih intuitif, terutama pada proses penggeseran elemen (insertion) dan penukaran nilai (swapping) yang melibatkan perubahan posisi data secara berulang.
2. Pembagian materi pra-praktikum melalui iLearn tetap perlu dilanjutkan agar mahasiswa memiliki fondasi konseptual yang matang tentang perbedaan SSL dan DLL sebelum menulis kode di laboratorium.

DAFTAR PUSTAKA

- [1] Oracle. "Trail: Creating a GUI With Swing". The Java™ Tutorials. 2024. Available:
<https://docs.oracle.com/javase/tutorial/uiswing/>